

Python OOP 深入：4 个你一定要搞懂的概念

前置：你已知道 class 怎么定义、self 是什么、继承用 `class Dog(Animal):`

目标：搞懂封装、@property、鸭子类型、魔法方法——它们解决什么问题、怎么用、常见的坑在哪

用时：每节 5-8 分钟，分 4 节

相关笔记：[高级OOP（抽象类/多重继承）](#) · [闭包与装饰器](#)

一、封装：Python 没有"不让碰"这个功能

先看问题

一个简单的银行账户：

```
class BankAccount:
    def __init__(self, owner, balance, password):
        self.owner = owner      # 户主
        self.balance = balance  # 余额
        self.password = password # 密码

acc = BankAccount("张三", 1000, "123456")
```

问题来了：`acc.password` 谁都看得见、谁都能改——但密码应该对外隐藏。

在大多数语言里，有一个叫 `private` 的关键字：`private` 的属性在类外面读不到、改不了。

但 Python 没有 `private`。一个属性写在类里，外面就能访问。

Python 的解决方案：靠约定，不靠强制

```
class BankAccount:
    def __init__(self, owner, balance, password):
        self.owner = owner      # 公开：谁都能碰
        self._balance = balance # _开头："内部使用，请勿直接修改"
        self.__password = password # __开头："Python 会改你的名字"

    def deposit(self, amount):
        if amount > 0:
            self._balance += amount
```

```
def get_balance(self):  
    return self._balance
```

三种写法的区别：

写法	意思	外面能访问吗？
self.owner	公开属性	✅ 能，随便读
self._balance	"这是内部细节，别碰"	⚠️ 能访问，但 IDE 会提示警告
self.__password	"别碰"	😬 看上去不能，但其实.....

_（单下划线）——纯粹的君子协定

```
acc = BankAccount("张三", 1000, "123456")  
print(acc._balance) # 1000——跑得通，但不应该这么用
```

_balance 和 owner 在 Python 眼里没有区别。加 _ 是在对读你代码的人说：这是内部实现细节，别直接动。你不听，Python 也不拦你。

__（双下划线）——名字篡改（name mangling）

```
acc = BankAccount("张三", 1000, "123456")  
  
print(acc.__password) # ❌ AttributeError: 'BankAccount' object has no attribute '__password'
```

哎？不是说不拦吗？——实际上 Python 偷偷改了名字：

```
print(acc.__dict__)  
# {'owner': '张三', '_balance': 1000, '_BankAccount__password': '123456'}
```

看到了吗？__password 被改成了 _BankAccount__password。所以：

```
print(acc._BankAccount__password) # ✅ "123456"——还是能读到
```

__ 的作用不是"禁止访问"，而是"改个名字，避免子类意外覆盖"。看个例子：

```
class Parent:  
    def __init__(self):
```

```

        self.__id = "parent"

    def get_id(self):
        return self.__id

class Child(Parent):
    def __init__(self):
        super().__init__()
        self.__id = "child"      # 这个 __id 和父类的 __id 不是同一个

c = Child()
print(c.get_id())              # "parent" (父类的 __id 没被覆盖)
print(c.__dict__)              # {'_Parent__id': 'parent', '_Child__id': 'child'}

```

两个 `__id` 分别变成了 `_Parent__id` 和 `_Child__id`，各不干扰。

一句话记

- `_xxx` = "请别动"——纯约定
- `__xxx` = "怕子类覆盖，改个名"——有机制但也不是真拦
- Python 从不拦你读任何属性——封装全靠自觉

破坏实验（猜完再跑）

```

class BankAccount:
    def __init__(self, owner, balance, password):
        self.owner = owner
        self._balance = balance
        self.__password = password

acc = BankAccount("李四", 2000, "654321")

# 猜：下面三行哪些能跑？哪些报错？
print(acc.owner)
print(acc._balance)
print(acc.__password)

```

二、@property：把方法伪装成属性

先看场景

```
class Circle:
    def __init__(self, radius):
        self.radius = radius

c = Circle(5)
print(c.radius)  # 5
```

一切正常。现在需求变了：半径不能为负数。

新手第一反应：加个 setter 方法

```
class Circle:
    def __init__(self, radius):
        self.radius = radius

    # 想加个校验方法
    def set_radius(self, value):
        if value < 0:
            raise ValueError("半径不能为负数")
        self.radius = value
```

但这有个问题：原来读的地方要改（从 `c.radius` 变成 `c.get_radius()`），写的地方也要改（从 `c.radius = -1` 变成 `c.set_radius(-1)`）。如果代码里已经用了 50 次 `c.radius`，全要改。

@property 的解法：不改调用方

```
class Circle:
    def __init__(self, radius):
        self._radius = radius  # 内部用 _radius 存

    @property
    def radius(self):           # 读的时候自动调用
        return self._radius

    @radius.setter
    def radius(self, value):    # 写的时候自动调用
        if value < 0:
            raise ValueError("半径不能为负数")
        self._radius = value

c = Circle(5)
print(c.radius)  # 5——读起来像属性，实际走了 @property
```

```
c.radius = 10      # 写起来像属性，实际走了 @radius.setter
c.radius = -1      # ❌ ValueError——校验生效了
```

调用方不需要改一行代码——`c.radius` 读和写的写法完全没变，但底层行为变了。

@property 的本质

`@property` 是一个装饰器（还记得装饰器吗？`@` 开头的那个东西），它的作用是：让一个方法在被调用时不用加括号。

```
class Order:
    def __init__(self, price, count):
        self._price = price
        self._count = count

    @property
    def total(self):
        """计算总价——每次读都重新算"""
        return self._price * self._count

order = Order(10, 3)
print(order.total)  # 30——像读属性一样，不加括号
```

@property 的三种用法

```
class Temperature:
    def __init__(self, celsius):
        self._celsius = celsius

    @property
    def celsius(self):          # 读: c.telsius → 返回数值
        return self._celsius

    @celsius.setter
    def celsius(self, value):  # 写: c.telsius = 30 → 走校验
        if value < -273.15:
            raise ValueError("低于绝对零度!")
        self._celsius = value

    @celsius.deleter
    def celsius(self):          # 删: del c.telsius → 打印+清空
        print(f"删除温度 {self._celsius}")
        del self._celsius

    @property
```

```
def fahrenheit(self):          # 只读属性——没有 setter
    return self._celsius * 9/5 + 32
```

```
t = Temperature(25)
print(t.celsius)              # 25
print(t.fahrenheit)           # 77.0——只读，计算出来的

t.celsius = 30                 # 走 setter
# t.fahrenheit = 100          # ❌ AttributeError——没有 setter，不能写

del t.celsius                  # 走 deleter
```

什么时候用 @property，什么时候直接暴露？

```
# ✅ 直接暴露：纯数据，不需要任何控制
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

# ✅ @property 只读：需要实时计算
class Order:
    def __init__(self, price, count):
        self._price = price
        self._count = count

    @property
    def total(self):
        return self._price * self._count

# ✅ @property 读写：需要控制写入
class Student:
    def __init__(self, name):
        self._name = name

    @property
    def name(self):
        return self._name

    @name.setter
    def name(self, value):
        if not isinstance(value, str):
            raise TypeError("名字必须是字符串")
        self._name = value
```

一句话记

先直接用属性，等需要控制时用 `@property` 升级——调用方不用改任何代码。

破坏实验

```
class Test:
    def __init__(self):
        self._x = 10

    @property
    def x(self):
        return self._x

    # 故意不写 setter

t = Test()
print(t.x)      # 能读到吗？
t.x = 20        # 能写进去吗？会报什么错？
```

三、鸭子类型 (Duck Typing)：只看行为，不看类型

一个场景

你有三个“解析器”：

```
class JsonParser:
    def parse(self, data):
        return f"JSON 解析：{data}"

class XmlParser:
    def parse(self, data):
        return f"XML 解析：{data}"

class YamlParser:
    def parse(self, data):
        return f"YAML 解析：{data}"
```

三个类没有继承关系、没有共同父类。但你想写一个函数来处理它们。

其他语言的做法

很多语言会说：你必须让这三个类"登记"一下——要么继承同一个父类，要么实现同一个接口。否则编译器不让你传进去。

Python 的做法

```
def process_data(parser, raw_data):  
    """我不关心 parser 是什么类型，能调 parse() 就行"""  
    return parser.parse(raw_data)  
  
process_data(JsonParser(), "{key: 'value'}") # ✓  
process_data(XmlParser(), "<root>value</root>") # ✓  
process_data(YamlParser(), "key: value") # ✓
```

Python 根本不检查类型。 它只做一件事：到运行时，调一下 `parser.parse(raw_data)` ——如果有 `parse` 方法就正常跑，没有就报错。

这就是"鸭子类型"名字的来源：

"如果它走路像鸭子、叫起来像鸭子，那它就是鸭子。"

更有意思的例子

```
class Duck:  
    def swim(self):  
        return "鸭子游"  
  
class Fish:  
    def swim(self):  
        return "鱼游"  
  
class ToyBoat:  
    def swim(self):  
        return "船浮着"  
  
def make_it_swim(thing):  
    return thing.swim()  
  
# 三个毫无关系的类，都传进去了  
make_it_swim(Duck())  
make_it_swim(Fish())  
make_it_swim(ToyBoat())
```

你可能会想：这不就是乱来吗？万一传了个石头进去呢？


```
class Stone:
    pass

make_it_swim(Stone()) # ❌ AttributeError: 'Stone' object has no attribute 'swim'
```

这就是鸭子类型的**优点也是缺点**：

- 优点：灵活，不需要提前设计接口体系，任何对象只要有需要的方法就能一起工作
- 缺点：错误到运行时才暴露，不会在写代码时告诉你

什么时候用鸭子类型，什么时候用继承？

```
# ✅ 鸭子类型：共享行为但不共享数据
class CsvReader:
    def read(self, path):
        print(f"读 CSV 文件: {path}")

class DbReader:
    def read(self, query):
        print(f"查数据库: {query}")

class ApiReader:
    def read(self, url):
        print(f"调接口: {url}")

def load_data(reader):
    return reader.read("some_source")

# ✅ 继承：共享数据 + 共享行为
class Animal:
    def __init__(self, name):
        self.name = name # 子类继承这个属性

    def eat(self):
        print(f"{self.name} 在吃东西")

class Dog(Animal):
    def bark(self):
        print(f"{self.name} 汪汪") # 用了父类的 self.name

class Cat(Animal):
    def meow(self):
        print(f"{self.name} 喵喵")
```

上手判断：如果几个类只有方法名相同（没有共用属性），用鸭子类型就够了。如果它们共享同样的属性（比如都有 `name`、`age`），用继承。

一句话记

Python 不检查对象"是什么类型"，只检查"有没有这个方法"。

破坏实验

```
def let_it_fly(thing):
    return thing.fly()

class Bird:
    def fly(self):
        return "鸟飞"

class Airplane:
    def fly(self):
        return "飞机飞"

class Stone:
    pass

print(let_it_fly(Bird()))      # 会怎样？
print(let_it_fly(Airplane())) # 会怎样？
print(let_it_fly(Stone()))    # 会怎样？
```

鸭子类型的另一面：函数也是鸭子

上面的例子都是"传对象，调方法"。但鸭子类型不止用于类——**函数参数也适用**。只要传进来的东西"能调用"，Python 就不管它是什么类型。

```
# 只要是可调用对象，run 一律接受
def run(func, *args, **kwargs):
    print("开始执行")
    result = func(*args, **kwargs)
    print("执行结束")
    return result

# 以下全是不同的"类型"，但都能传进去
def say_hello(name):          # 普通函数
    return f"你好, {name}!"

run(say_hello, "李四")        # 
run(lambda x: x * 2, 5)       #  lambda
```

```
run(print, "hello") #  内置函数

class Greeter:      # 带 __call__ 的对象
    def __call__(self, name):
        return f"你好, {name}!"

run(Greeter(), "王五") #  对象也能当函数用
```

核心： `run()` 不检查 `func` 的类型，只做一件事——`func()`。这和 `process_data(parser)` 不检查 `parser` 类型、只调 `parser.parse()` 是一模一样的逻辑。

callable() — 提前问一下"你能调吗"

```
callable(print)      # True - 内置函数，能调
callable(lambda: 1)  # True - lambda，能调
callable("hello")    # False - 字符串不能调
callable(Stone())    # False - 石头不能调

# 用法：先问再调，避免崩溃
def safe_run(func, *args):
    if callable(func):
        return func(*args)
    else:
        print(f"{func} 不可调用")
```

Python 内置到处是鸭子类型

你写过的	鸭子类型在哪
<code>sorted(words, key=len)</code>	<code>key</code> 只要能用 <code>key(元素)</code> 就行，不看类型
<code>map(str, nums)</code>	<code>str</code> 是类名，但能调就传
<code>@app.get("/")</code>	FastAPI 把函数当参数存，请求来了就调
<code>" ".join(iterable)</code>	参数只要可迭代就行， <code>list/tuple/set</code> 都能传

四、魔法方法：为什么 `print(obj)` 能打印出内容

你每天都用，但没注意到

```
class Book:
    def __init__(self, title, author, pages):
```

```
        self.title = title
        self.author = author
        self.pages = pages

b = Book("三体", "刘慈欣", 400)
print(b)      # <__main__.Book object at 0x...>
```

`print(b)` 打印出来的是"这个对象在内存的地址"，读不懂。

但如果你想要这个样子：

```
print(b)      # 《三体》 - 刘慈欣
```

怎么办？——定义一个 `__str__` 方法。

`__str__`：控制 `print` 输出

```
class Book:
    def __init__(self, title, author, pages):
        self.title = title
        self.author = author
        self.pages = pages

    def __str__(self):
        return f"《{self.title}》 - {self.author}"

b = Book("三体", "刘慈欣", 400)
print(b)      # 《三体》 - 刘慈欣
```

`__str__` 是 Python 内置的"钩子"——当你 `print(obj)` 或 `str(obj)` 时，Python 自动去调对象的 `__str__()` 方法。不定义的话，就用默认的（打印内存地址）。

魔法方法是什么

以 `__` 开头和结尾的方法，统称为**魔法方法**（**magic methods**）。它们不是让你直接调用的——是 Python 在背后自动调用的。

你写：

```
print(b)
```

Python 实际做：

```
b.__str__() # 自动调这个
```

常用的魔法方法清单

```
class Book:
    def __init__(self, title, author, pages):
        self.title = title
        self.author = author
        self.pages = pages

    # ---- 字符串相关 ----
    def __str__(self):
        """print(obj) 和 str(obj) 时调用"""
        return f"《{self.title}》- {self.author}"

    def __repr__(self):
        """调试时调用，交互式环境直接输入 obj 时也调"""
        return f"Book(title='{self.title}', author='{self.author}')"

    # ---- 比较相关 ----
    def __eq__(self, other):
        """obj1 == obj2 时调"""
        return self.title == other.title and self.author == other.author

    def __lt__(self, other):
        """obj1 < obj2 时调，用在排序"""
        return self.pages < other.pages

    # ---- 容器相关 ----
    def __len__(self):
        """len(obj) 时调"""
        return self.pages

    def __bool__(self):
        """if obj: 时调"""
        return self.pages > 0
```

——验证：

```
b1 = Book("三体", "刘慈欣", 400)
b2 = Book("三体", "刘慈欣", 400)
b3 = Book("小王子", "圣埃克苏佩里", 96)

print(b1) # 《三体》- 刘慈欣 ← __str__
```

```

print(repr(b1))          # Book(title='三体', ...)    ← __repr__
print(b1 == b2)          # True                      ← __eq__
print(b1 < b3)           # False (400 > 96)          ← __lt__
print(len(b1))           # 400                      ← __len__
print(bool(Book("空", "无", 0))) # False (pages=0)   ← __bool__

# 排序也靠 __lt__
books = [b3, b1]
books.sort()
print([str(b) for b in books])

```

__str__ vs __repr__ 的区别

```

class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __repr__(self):
        """给人看"这个对象是什么"——最好能直接复制出来用"""
        return f"Point({self.x}, {self.y})"

    def __str__(self):
        """给用户看"这个对象的内容"——友好的展示"""
        return f"({self.x}, {self.y})"

p = Point(3, 4)
print(p)          # (3, 4)      ← 用户友好    ← __str__
str(p)            # (3, 4)      ← __str__
repr(p)           # Point(3, 4) ← __repr__
f"{p}"            # (3, 4)      ← __str__
f"{p!r}"          # Point(3, 4) ← 加 !r 强制用 __repr__

```

开发阶段的黄金法则：定义 `__repr__` 就够了——如果没有 `__str__`，Python 会用 `__repr__` 代替。但 `__str__` 优先给 `print()` 用。

__eq__ 和 __hash__ 必须配对的坑

```

class User:
    def __init__(self, name):
        self.name = name

    def __eq__(self, other):
        return self.name == other.name

```

```

u1 = User("张三")
u2 = User("张三")
print(u1 == u2)  # True ✅

# 但放进 set 试试:
users = {u1, u2}
print(len(users))  # 2? 不是 1? 两个张三按理应该去重才对

```

原因：set 判断重复时，先比 `hash`（哈希值），再比 `__eq__`。你重写了 `__eq__` 但没有重写 `__hash__`，Python 用默认的 `__hash__`（基于内存地址），所以 `u1` 和 `u2` 的 `hash` 值不同，被认为不是同一个对象。

修复：

```

class User:
    def __init__(self, name):
        self.name = name

    def __eq__(self, other):
        return isinstance(other, User) and self.name == other.name

    def __hash__(self):
        return hash(self.name)  # hash 值基于 name 计算

u1 = User("张三")
u2 = User("张三")
users = {u1, u2}
print(len(users))  # 1 ✅ 同一个 hash，再比 eq 发现相等，只保留一个

```

规则：重写 `__eq__` 就必须重写 `__hash__`，否则 `set` 和 `dict` 的行为会出乎意料。

一句话记

魔法方法是 Python 留给你的"钩子"：`print(obj)` → 调 `__str__`，`obj == other` → 调 `__eq__`，`len(obj)` → 调 `__len__`。不定义就用默认行为。

破坏实验

```

class BadHash:
    def __init__(self, val):
        self.val = val

    def __eq__(self, other):

```

```
        return self.val == other.val

# __hash__ 没写

b1 = BadHash(1)
b2 = BadHash(1)
print(b1 == b2)    # True? False?
d = {b1: "value"}  # 这句会报错吗？
```

五、三种方法：实例方法 vs 类方法 vs 静态方法

先看一个场景

你写了一个 `Student` 类：

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

    def info(self):                    # ← 实例方法
        return f"{self.name}: {self.score}分"
```

这个 `info(self)` 是**实例方法**——必须创建对象才能调：

```
s = Student("张三", 92)
print(s.info())    # ✅ 对象调实例方法
```

但有些场景实例方法解决不了：

场景 1：我想知道"全班及格线是多少"——这个不依赖某个学生，而是跟班级整体有关。

场景 2：我想把 "张三,92" 这个字符串直接转成一个 `Student` 对象——这是个"从别处创建 `Student`"的辅助功能。

实例方法（`self`）不适合这两种情况。于是有了**类方法**和**静态方法**。

三种方法对比

```
class Demo:
    # — 实例方法 —
    def instance_method(self):
        """最普通的方法，必须实例化才能调"""
```



```
        return f"实例方法, self={self}"

# — 类方法 —
@classmethod
def class_method(cls):
    """通过类调, 第一个参数是类本身"""
    return f"类方法, cls={cls}"

# — 静态方法 —
@staticmethod
def static_method():
    """就是个普通的函数, 恰好放在类里"""
    return "静态方法, 没有 self 也没有 cls"
```

调用方式：

```
obj = Demo()

# 实例方法：只能对象调
print(obj.instance_method())    # ✓
# print(Demo.instance_method()) # ✗ 类不能直接调实例方法（除非传对象）

# 类方法：类和对象都能调
print(Demo.class_method())      # ✓ 推荐用法
print(obj.class_method())       # ✓ 也能调，但不常见

# 静态方法：类和对象都能调
print(Demo.static_method())     # ✓ 推荐用法
print(obj.static_method())      # ✓ 也能调
```

核心区别一张表

	实例方法	类方法	静态方法
装饰器	无	@classmethod	@staticmethod
第一个参数	self（实例本身）	cls（类本身）	无特殊参数
能访问实例属性？	✓ 能	✗ 不能	✗ 不能
能访问类属性？	✓ 能（通过 self.__class__）	✓ 能（通过 cls）	✗ 不能
调用方式	obj.method()	Class.method()	Class.method()

	实例方法	类方法	静态方法
什么时候用	绝大多数情况	操作和类相关但不依赖具体实例	和类有关但不需要任何类数据

类方法的经典用法：工厂方法

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

    @classmethod
    def from_string(cls, data: str):
        """把 '张三,92' 这样的字符串转成 Student 对象"""
        name, score = data.split(",")
        return cls(name, float(score))    # cls() = Student()

    @classmethod
    def from_file(cls, filepath: str):
        """从文件批量读取学生"""
        students = []
        with open(filepath, "r", encoding="utf-8") as f:
            for line in f:
                students.append(cls.from_string(line.strip()))
        return students

    def info(self):
        return f"{self.name}: {self.score}分"

# 正常创建
s1 = Student("张三", 92)

# 通过类方法创建
s2 = Student.from_string("李四,88")
print(s2.info())          # 李四: 88.0分

# 批量从文件创建（如果 students.txt 存在）
# students = Student.from_file("students.txt")
```

关键点： `from_string` 里用了 `cls` 而不是硬编码 `Student`。这意味着子类继承时也能正常工作：

```
class CollegeStudent(Student):
    pass

cs = CollegeStudent.from_string("王五,75")
print(type(cs).__name__) # CollegeStudent (不是 Student!)
```

如果用 `return Student(...)` 硬编码，子类调用就返回父类对象了。

静态方法的经典用法：工具函数

```
class ScoreUtils:
    """分数相关工具"""

    @staticmethod
    def is_pass(score: float) -> bool:
        """判断是否及格"""
        return score >= 60

    @staticmethod
    def grade(score: float) -> str:
        """把分数转等级"""
        if score >= 90: return "A"
        if score >= 80: return "B"
        if score >= 70: return "C"
        if score >= 60: return "D"
        return "F"

    @staticmethod
    def average(scores: list) -> float:
        """算平均分"""
        return sum(scores) / len(scores)

# 静态方法不需要创建对象，直接类名.方法名
print(ScoreUtils.is_pass(85)) # True
print(ScoreUtils.grade(85)) # B
print(ScoreUtils.average([90, 80, 70])) # 80.0
```

和普通函数的区别：`ScoreUtils.is_pass(85)` 比 `is_pass(85)` 更有条理——通过类名就知道这个函数跟“分数”有关。

综合对比实战

```

class Employee:
    company = "ABC 科技"          # 类属性
    count = 0                     # 类属性: 员工总数

    def __init__(self, name, salary):
        self.name = name          # 实例属性
        self.salary = salary      # 实例属性
        Employee.count += 1       # 每创建一人, 总数 +1

# 实例方法: 和具体员工相关
def info(self):
    return f"{self.name} 在 {self.company}, 薪资 {self.salary}"

# 类方法: 和类相关, 但不依赖具体员工
@classmethod
def from_annual(cls, name: str, annual: float):
    """从年薪创建 (按月薪存)"""
    return cls(name, annual / 12)

@classmethod
def total_employees(cls):
    return f"共有 {cls.count} 名员工"

# 静态方法: 和员工有关但不需要任何类/实例数据
@staticmethod
def validate_salary(amount: float) -> bool:
    return amount > 0

e1 = Employee("张三", 8000)
e2 = Employee.from_annual("李四", 240000) # 年薪24万 → 月薪2万

print(e1.info())                  # 张三 在 ABC 科技, 薪资 8000    ← 实例方法
print(Employee.total_employees()) # 共有 2 名员工                ← 类方法
print(Employee.validate_salary(-500)) # False                    ← 静态方法

```

三句话记

1. **实例方法** (`self`): 90% 的情况用这个, 需要访问具体对象的数据
2. **类方法** (`cls`): 工厂方法、操作类属性, 不依赖具体实例
3. **静态方法** (无特殊参数): 工具函数, 放在类里为了组织代码

破坏实验

```

class Test:
    @classmethod
    def cm(cls):
        return f"cls = {cls.__name__}"

    @staticmethod
    def sm():
        return "静态方法"

# 猜结果:
print(Test.cm())      # ?
print(Test.sm())      # ?

# 用实例调类方法和静态方法:
t = Test()
print(t.cm())         # ?
print(t.sm())         # ?

# 如果静态方法想访问类属性?
class Bad:
    @staticmethod
    def wrong():
        return company    # 能访问到吗?

class Good:
    company = "ABC"

    @staticmethod
    def right():
        return Good.company    # 这样写可以吗?

```

总结

内容	一句话	最需要注意的
封装	<code>_</code> 是约定, <code>__</code> 是改名, 都不拦你	Python 没有真 private, 全凭自觉
@property	把方法伪装成属性, 调用方不用改	没写 setter 的属性不能赋值
鸭子类型	"有这个方法就行", 不看类型	错误到运行时才暴露
魔法方法	<code>__xxx__</code> 是 Python 自动调的钩子	<code>__eq__</code> 和 <code>__hash__</code> 必须一起重写
三种方法	实例(<code>self</code>)→对象数据	类(<code>cls</code>)→工厂/类属性

备考相关

- [EXAM_PREP/day04/00_今日任务](#) — Day 4 面向对象
- [EXAM_PREP/day06/00_今日任务](#) — Day 6 老师课堂代码重放